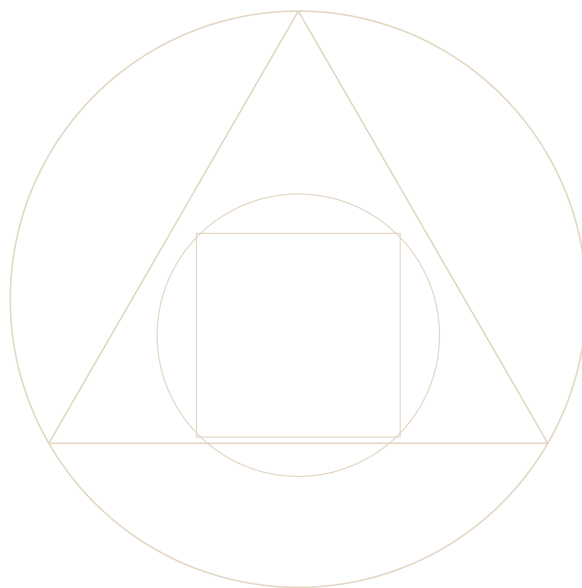




A COMPOUND REASONING SYSTEM ON ONE CONSUMER GPU

Eirenaeus-*Philalethes*

The same open model, used better: equal quality to the base model alone, measurably faster, and able to operate a computer.

1.51×

faster in total wall-clock time

677

problems across four public benchmarks

$p \geq 0.62$

no significant quality difference on any benchmark

12 GB

one consumer GPU, temperature 0, same harness

Selection over generation · Listen, think, act · Measure before claiming

AUTHOR

J.C. - Asastu

PLACE

Buenos Aires

BASE MODEL

Ternary-Bonsai-2-27B

ABSTRACT

The same weights, a better way to use them

Eirenaeus-Philalethes is a local compound AI system. It wraps an open 27-billion-parameter reasoning model with a small decision layer that runs on the CPU, a supervisor over the model's thinking, a pair of hands that can operate the computer, and a verified memory. It never changes the model's weights.

On 677 problems from four public benchmarks (HumanEval+, MBPP+, AIME 2025 and a held-out mix of code and math), measured on the same machine at temperature 0 against the same base model served alone, Eirenaeus shows **no statistically significant difference in quality** (McNemar exact test, $p \geq 0.62$ on every benchmark) while being **1.51× faster in total wall-clock time** (1.27× to 1.93× depending on the task). On tool calling (BFCL v4, 400 cases) it ties the base model in quality and gives no speed advantage; I report that too. It also does what the base model cannot do alone: it operates a computer.

Everything here was measured on a single RTX GPU with 12 GB of memory. I report what worked, what did not, and the mistakes in my own measurement that I caught along the way, because a claim is only as good as the method behind it.

1.51×

faster overall

512 min vs 771 min

1.93×

best case

HumanEval+, 156 vs 154 solved

0significant quality losses
across 677 paired problems**~0.5 s**decision on the CPU
before any GPU work starts

KEY FINDINGS

- 01 Same quality.** No benchmark shows a significant difference; on HumanEval+ Eirenaeus solves two more.
- 02 Faster where it matters.** The gain grows with how long the core would think: up to 1.93× on code.
- 03 No edge on tool calls.** Short calls need no thinking; the base model with thinking off is just as fast.
- 04 Method over luck.** One noisy run once suggested a loss that did not exist; every claim here is paired, at temperature 0.

CONTENTS

1	Motivation	3	6	Results: tool calling	7
2	Architecture	3	7	What did not work	8
3	Design principles	4	8	Safety: the hands are denied by default	8
4	Evaluation methodology	4	9	Limitations, roadmap and reproducibility	9
5	Results: reasoning	5			

SECTION 1

Motivation

A strong open model on a consumer GPU spends most of its time thinking. Much of that thinking is either unnecessary, because the answer was clear early, or misdirected, because a simple request was treated as a hard one.

A system that decides *how much* to think before thinking, and notices *when* it can stop, should save time without losing answers. The question I set out to answer honestly is narrow on purpose: **can an orchestration layer around a fixed model make it faster without making it worse?** Not a better model. The same weights, used better.

SECTION 2

Architecture

The system is named after the three alchemical principles and the fifth essence. The names are a mnemonic, not a claim. A request reaches an OpenAI-compatible gateway; the decision layer reads it in about half a second and sends it down one of three paths.

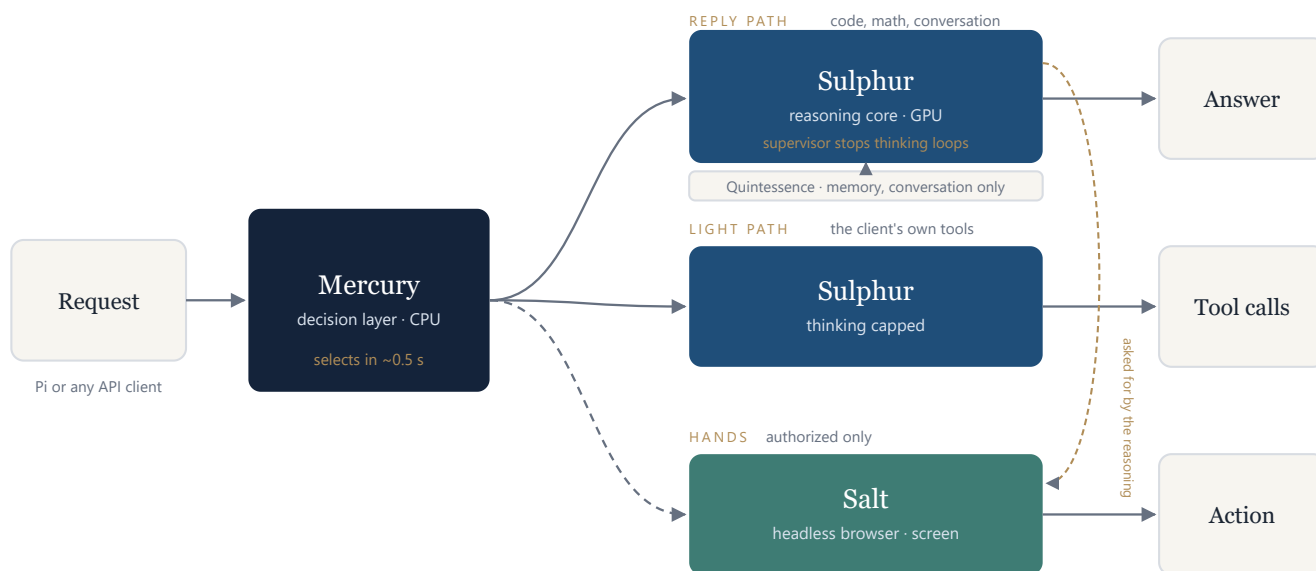


FIGURE 1 Request flow. The decision layer only selects; the reasoning core does the thinking; the hands act only when the reasoning asks and every safety check agrees.

PART	ROLE	RUNS ON
Mercury · decision layer	Laya, 421M parameters. Selects how much to think, whether the request is code, needs live data, or is an order for the computer.	CPU
Sulphur · reasoning core	Ternary-Bonsai-2-27B, PQ2_0, served by llama.cpp at about 65 tokens per second.	GPU · 10 GB
Salt · hands	Operate the computer: a headless browser by default, the real screen only when authorized.	CPU
Quintessence · memory	Its own small store, inspired by Engram (Gentleman Programming) but injected rather than searched: chosen on the CPU, no model tokens spent. Validity follows Dokimos : every memory expires and is re-attested against its source.	CPU

Three paths. The *reply path* covers code, math and conversation: the core thinks with the effort the decision layer chose, and a supervisor watching the thinking stream stops loops (the same doubt repeated three times) by asking for the answer with what it has. Requests whose answer is the reply itself never receive memory and are never offered the computer. The *light path* serves clients that bring their own tools: only the effort pick and a thinking cap. The *hands* act only when the reasoning asks and every safety check agrees.

SECTION 3

Design principles

I

Selection over generation

The decision layer never writes text; it chooses among options. Choosing is cheap, about 0.4 s on a CPU, and checkable; every escalation to the core becomes a training label.

II

Listen, think, act

Like a person: understand the request, think, and only then act on the computer. The hands are a tool the reasoning asks for, never the first move.

III

Measure before claiming

Every claim comes from a per-item results file: never from a console, never from one lucky run, and always against a control.

SECTION 4

Evaluation methodology

Same machine, same harness

Both arms answer the same questions with the same client and checker: Eirenaeus against the base model served alone with its authors' default settings.

Paired statistics

McNemar's exact test on the problems where the two arms disagree. "Better" or "worse" only below $p = 0.05$; otherwise, "no difference".

The control arm

When Eirenaeus seems to win through a setting, the base model is run with that setting too. If it matches, the credit goes to the setting.

Temperature 0

A single run with randomness is not evidence. Figure 2 shows a loss that was pure noise.

Held-out and judge splits

105 problems development never used; agentic work keeps a judge split run once per milestone and never tuned on.

An unshared GPU

Every step logs memory use and any other GPU process; a shared GPU voids the run.

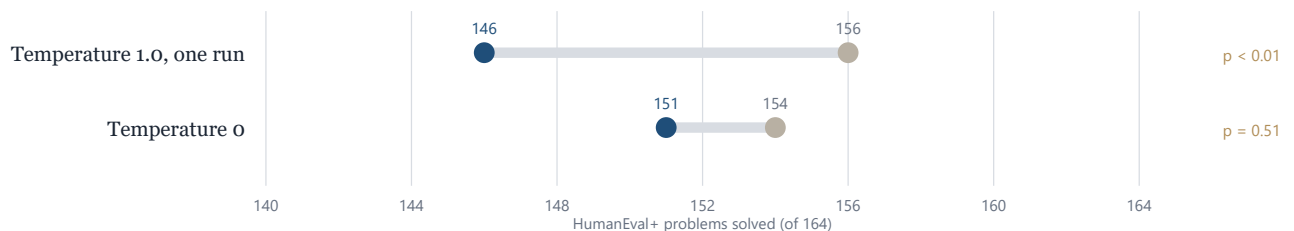


FIGURE 2 The noise lesson. One run at temperature 1.0 suggested Eirenaeus lost ten HumanEval+ problems ($p < 0.01$). At temperature 0 the gap disappeared ($p = 0.51$). I had already changed the system to fix a problem that did not exist.

SECTION 5

Results: reasoning

Across 677 paired problems, Eirenaeus solves as many as the base model alone and finishes the whole set in 512 minutes instead of 771: 1.51× faster, with no benchmark showing a significant difference in quality.

BENCHMARK	ITEMS	EIRENAEUS	BASE	ONLY E / ONLY BASE	MCNEMAR P	TIME E	TIME BASE	SPEED
Held-out mix	105	97	98	1 / 2	1.00	6,604 s	8,561 s	1.30×
MBPP+	378	308	309	7 / 8	1.00	12,227 s	19,228 s	1.57×
AIME 2025	30	27	27	1 / 1	1.00	6,827 s	8,691 s	1.27×
HumanEval+	164	156	154	3 / 1	0.62	5,055 s	9,752 s	1.93×
Total	677					30,712 s	46,231 s	1.51×

Temperature 0, one RTX GPU with 12 GB, Eirenaeus v2 against Ternary-Bonsai-2-27B served alone with its default thinking budget. "Only E / only base" counts the problems one arm solved and the other did not; those discordant pairs are what the McNemar test reads.

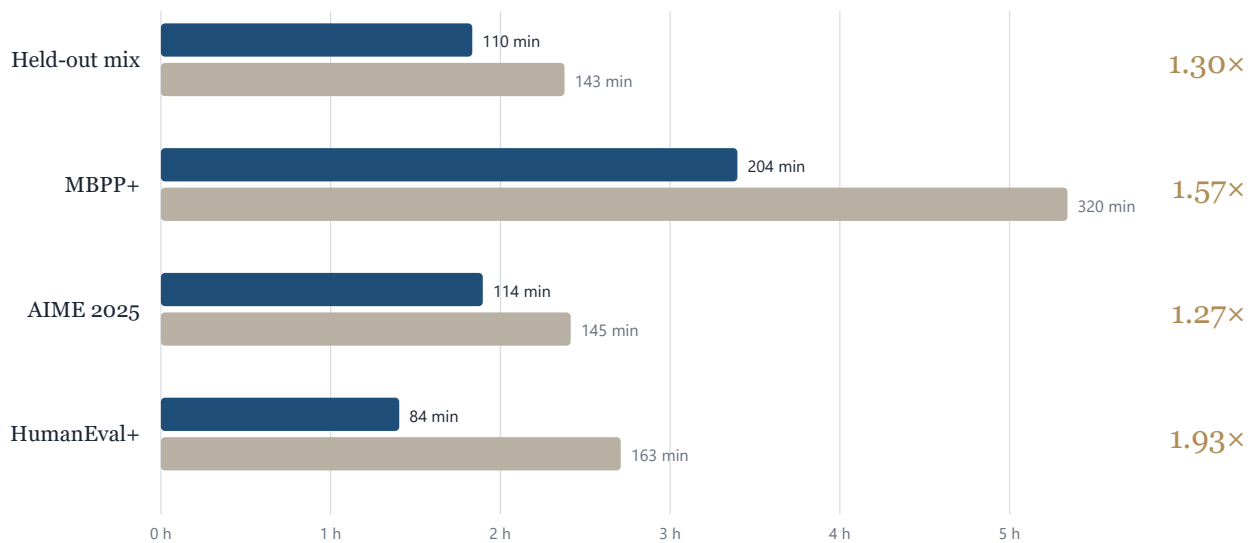


FIGURE 3 Wall-clock time per benchmark. Dark bars: Eirenaeus. Light bars: the base model alone. The gain is largest where the core would otherwise think the longest.

Where the time goes. On short problems Eirenaeus carries a fixed cost of one to two seconds per request (median 12.3 s against 11.2 s on the held-out mix). It wins that back many times over on problems where the core would think for minutes: the supervisor cuts loops, and code is no longer offered tools it would only be refused.

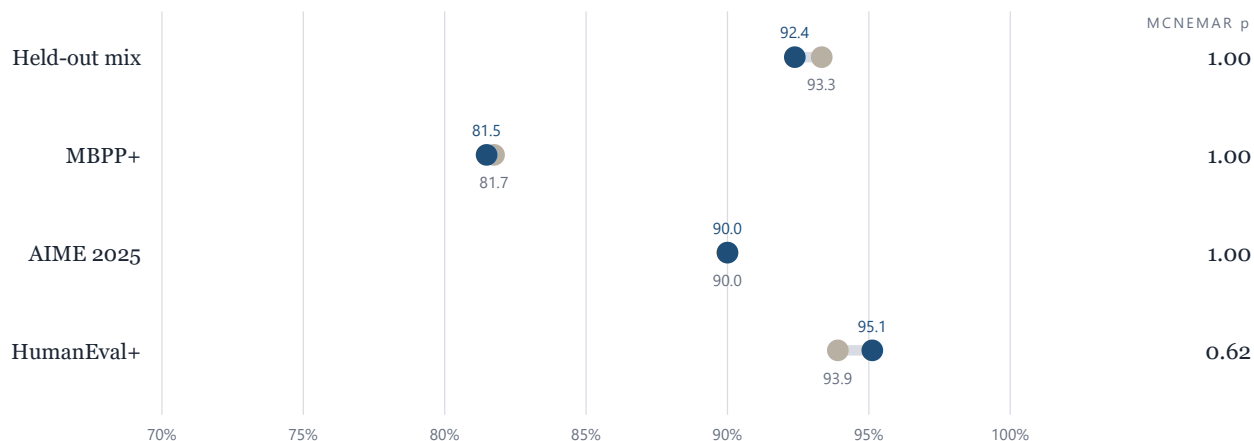


FIGURE 4 Accuracy per benchmark. Dark: Eirenaeus. Light: the base model alone. The right column is the McNemar p-value; every pair overlaps within noise.

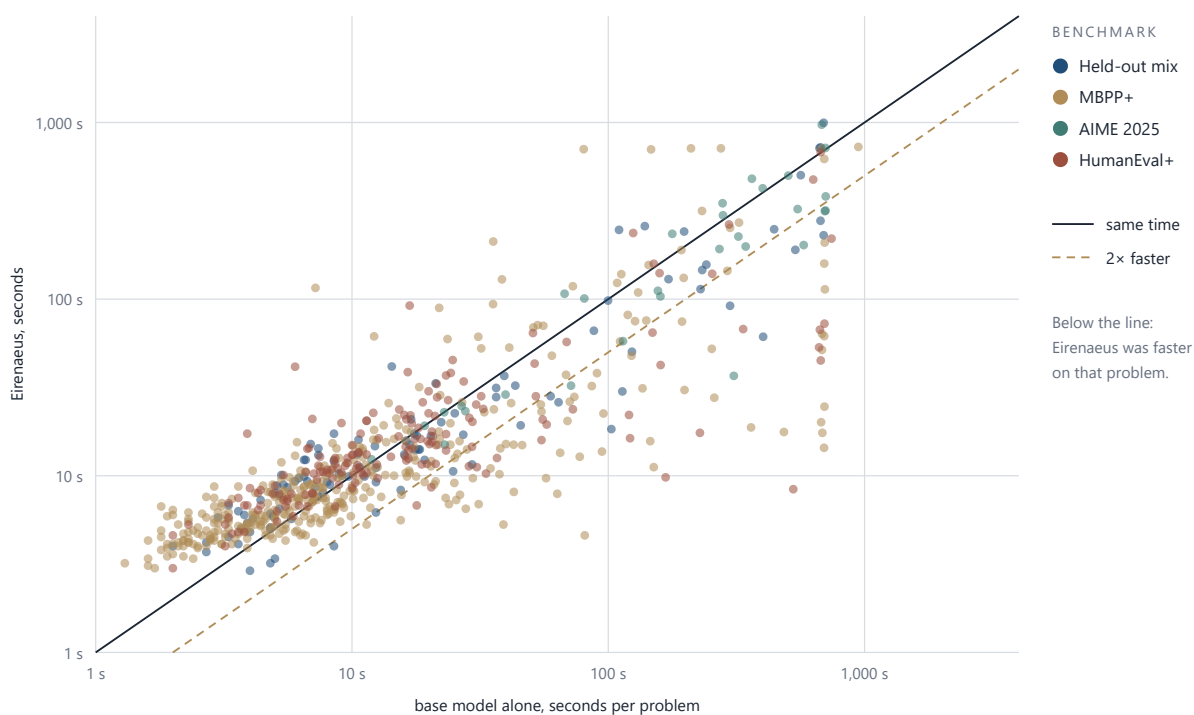


FIGURE 5 Every one of the 677 problems. Each dot compares the time the base model took (horizontal) with the time Eirenaeus took (vertical), on log scales. The cloud sits below the diagonal on long problems and just above it on very short ones.

What the speedup is

A property of the orchestration around one fixed model on one machine: deciding how much to think, stopping loops, and not offering tools that would only be refused. The weights are identical.

What it is not

A faster model, or a result on other hardware. On very short requests the decision layer is pure overhead, which is why tool calling, next, shows no advantage.

SECTION 5.2

Results: tool calling

On BFCL v4 (simple, multiple, parallel and parallel-multiple calls, 100 each, official AST checker, temperature 0) Eirenaeus ties the base model in quality but brings no speed advantage. The control arm shows why.

CONFIGURATION	CORRECT	VS REFERENCE	TOTAL TIME	MEDIAN
Base alone, thinking (as published)	373 / 400	reference	1,589 s	3.1 s
Base alone, thinking off	371 / 400	$p = 0.84$	877 s	1.8 s
Eirenaeus, all layers	362 / 400	worse, $p = 0.04$	3,649 s	5.1 s
Eirenaeus, light path	375 / 400	tie, $p = 0.75$	2,027 s	3.8 s
Eirenaeus, light path, thinking off	368 / 400	$p = 0.42$	1,072 s	1.8 s

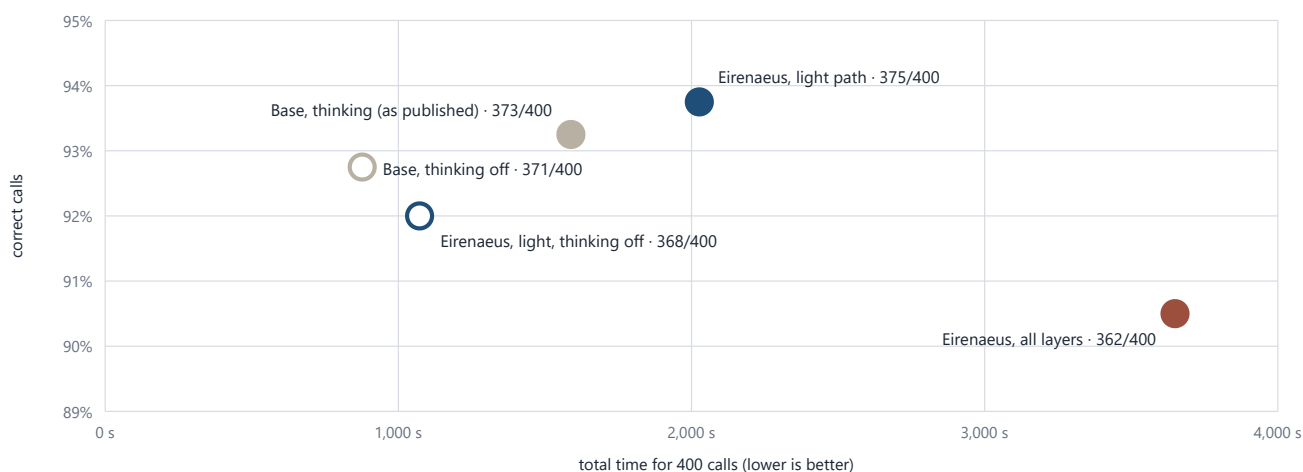


FIGURE 6 Accuracy against total time for 400 calls. Hollow markers: thinking off. Tool calls barely need thinking, so the fastest configuration is simply the base model with thinking off.

Reading the control. With every layer on, Eirenaeus was worse ($p = 0.04$): the date line in its prompt turned "this season" into the wrong year, and a live-data note leaked into unrelated calls. The light path restores quality (375 against 373). Turning thinking off makes everyone fast, the base model most of all, so that speed belongs to the setting, not to Eirenaeus. I report it as a tie.

5.3 Agentic work

The base model alone cannot operate a computer, so there is no paired comparison to report. My agentic evaluation, where every task is checked against the world rather than against the agent's own report, is in progress and will be published once it runs on a public benchmark.

SECTION 5.4

Results: vision, and the current version

Images were the one place where Eirenaeus lost clearly. The cause sat in the decision layer, not in the core: it read only the words of a request, so an exam question with a figure looked like a short factual answer and got no thinking at all.

BENCHMARK	ITEMS	EIRENÆUS	BASE	ONLY E / ONLY BASE	MCNEMAR P	TIME E	TIME BASE	SPEED
MMMU-Pro set 1 · v2	200	129	145	7 / 23	< 0.01	11,021 s	9,318 s	0.85×
MMMU-Pro set 2 · v3	200	130	137	9 / 16	0.23	7,256 s	7,848 s	1.08×
HumanEval+ · v3	164	156	154	3 / 1	0.62	5,014 s	9,752 s	1.94×
Held-out mix · v3	105	100	98	2 / 0	0.50	6,311 s	8,561 s	1.36×

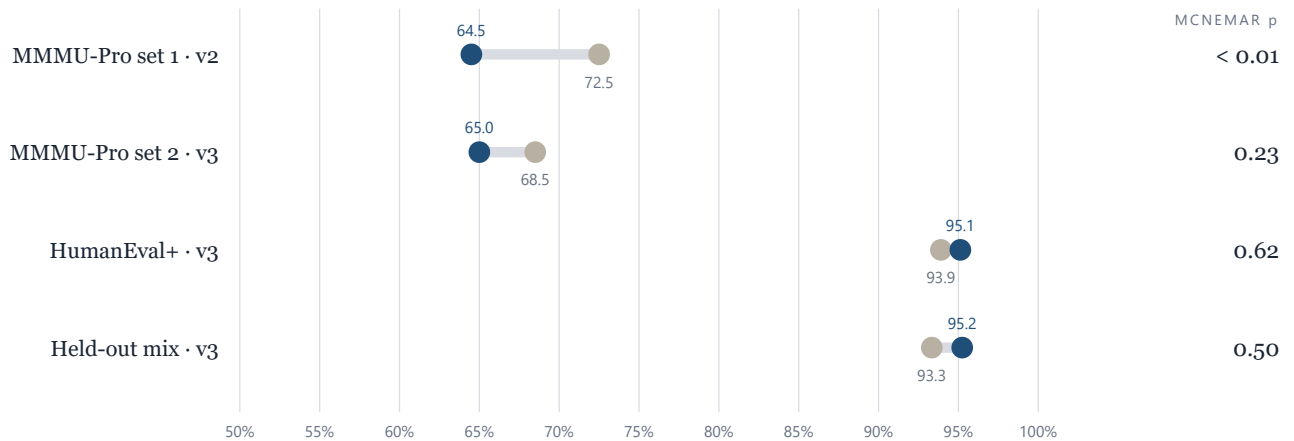


FIGURE 7 Accuracy, Eirenaeus (dark) against the base model alone (light). Set 1: 200 MMMU-Pro questions, v2. Set 2: 200 new questions never used in development, v3. Below them, the two reasoning benchmarks re-run with v3.

What changed in v3. The decision layer now receives a profile of the request (how many images it carries, whether tools are offered) and a request with images never goes to "no thinking". An exam answer with images also travels without Eirenaeus's own context. On set 1, v2 solved 16 fewer than the base model ($p < 0.01$, significant). On set 2, v3 solves 7 fewer ($p = 0.23$): no significant difference, and no advantage either.

A fix that broke the speed

The first v3 removed Eirenaeus's context from every direct answer, code included. HumanEval+ fell to 0.66× the base model's speed: one problem took 691 s instead of 22. That context is what keeps the core's thinking short. Limited to images, the speed came back (1.94×).

The current version

v3 was re-run on HumanEval+ and the held-out mix: same quality as the base model (156 vs 154, 100 vs 98) at 1.94× and 1.36×. MBPP+ and AIME 2025 were not re-run; the table in Section 5 is v2.

SECTION 6

What did not work

These are as informative as the wins, and each one changed the design.

A goal loop inside the planner

Stating the goal and re-planning until it was met changed the plans themselves; tasks that already worked started failing, and a goal whose only evidence was a file name reported "checked" on a wrong answer. A cleaner second version showed no significant gain at 61% more time.

Reasoning before acting

A planner that thinks before handing steps to the hands showed no gain on held-out orders, at 77% more time, and read an idiomatic request ("play something") as a purchase.

Offering tools that would be refused

Asked to write code, the model kept requesting the computer to "test" it; each refusal cost a new round of thinking, and one problem took 47 minutes. Removing the offer removed the loop.

Fixing a problem that did not exist

A noisy single run suggested a code-quality loss; sending all code to maximum thinking cost the code speedup. The temperature-0 rerun showed there had been no loss (Figure 2).

SECTION 7

Safety: the hands are denied by default

During one benchmark the agent operated my real screen: it typed into the file explorer, because the benchmark client did not request headless mode and a generic word ("file") counted as a request for the computer. Every benchmark now forces headless mode, and the design moves from *allowed unless filtered* to **denied unless authorized**.

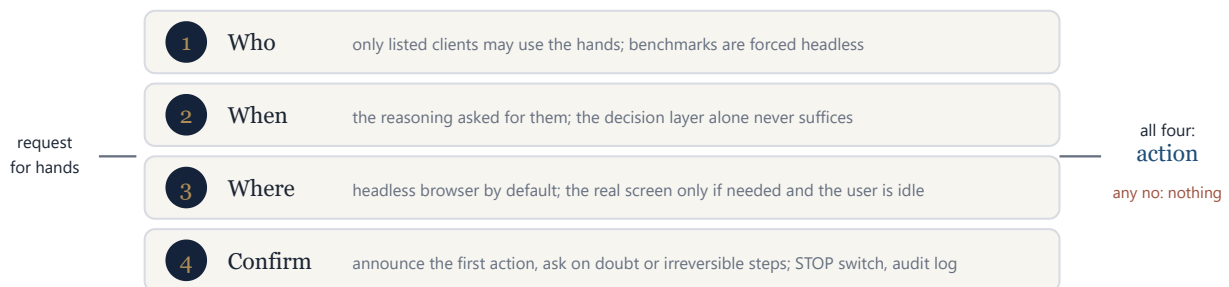


FIGURE 8 Four checks stand between a request and the real screen. A set of requests where the agent must not act has to produce zero actions on every change.

BEFORE	NOW
The hands were allowed unless a filter stopped them.	The hands are denied unless every check authorizes them.
A benchmark client could reach the real screen.	Benchmarks and practice are forced headless by the server.
A generic word ("file", "screen") counted as a request.	Only a concrete target on the computer counts: a site, an app, a folder.
Code was offered the computer and refused it in loops.	A request answered in the reply is never offered the computer.

SECTION 8

Limitations

- **One machine, one GPU class, one base model.** The speedup belongs to this setup until measured elsewhere.
- **Development touched public benchmarks.** The held-out mix and judge split guard against overfitting to them.
- **Short requests carry overhead.** The decision layer costs about half a second per request.
- **Computer use needs training on the ground.** Clear orders work today; quality in each application needs the decision layer post-trained there, on verified episodes (Roadmap).
- **No advantage on images.** On MMMU-Pro it is within noise of the base model (130 against 137).

SECTION 9

Roadmap

NEXT Verified early stop for code Stop thinking as soon as the code passes the examples in its own prompt; keep thinking only when it fails.	NEXT Hands authorization The four checks of Section 7, with a must-not-act set as a permanent regression test.	THEN A public agentic benchmark Agentic results reported on a benchmark others can run.	THEN Per-application post-training Train the decision layer from verified episodes, one application at a time.
--	--	---	--

SECTION 10

Reproducibility

Hardware	One NVIDIA RTX GPU with 12 GB of VRAM, a consumer desktop CPU, Windows 11.
Base model	Ternary-Bonsai-2-27B, PQ2_0, llama.cpp, 64K context, reasoning budget 36,864 tokens.
Decision layer	Laya, 421M parameters, on the CPU.
Benchmarks	HumanEval+ and MBPP+ (EvalPlus v0.2.0), AIME 2025, BFCL v4 with its official AST checker, and a held-out mix with a fixed offset that development never saw.
Evidence	Every number comes from a per-item results file (id, pass, seconds, route), available on request; every figure is generated from them.

HOW TO CITE

J.C. - Asastu. Eirenaeus-Philalethes: a compound reasoning system on a single consumer GPU. Whitepaper v0.1, Buenos Aires, October 2026.



Eirenaeus-Philalethes · J.C. - Asastu · Buenos Aires, October 2026. Set in Georgia and Segoe UI; figures generated from the result files at build time.

Acknowledgements. Memory inspired by Engram by Gentleman Programming (github.com/Gentleman-Programming/engram); validity rules after Dokimos (github.com/Jc-asastu/dokimos).